

Modeling Team Performance in NCAA Division I Women's Ice Hockey Using Elo Ratings

Taylor Lum

Honors in Data Science 2024/2025

Advisors: Dr. Robin Lock & Dr. Ivan Ramler

Department of Mathematics, Computer Science, and Statistics

St. Lawrence University

Abstract

Pairwise Rankings (PWR) determine NCAA Division I women's ice hockey postseason tournament selections. PWR uses a metric called Rating Percentage Index (RPI), head-to-head results, and record against common opponents. However, PWR has limitations, including its reliance on aggregate statistics, strength of schedule biases, and binary win/loss outcomes. This project explores Elo ratings, a system originally developed for ranking chess players, as an alternative, offering a dynamic system that updates based on game results while incorporating margin of victory and home ice advantage. By evaluating Elo's ability to model team performance and predict outcomes, this study assesses its viability as a meaningful rating system for ice hockey. Additionally, we implement a web app that automatically displays up-to-date ratings that are also used to make predictions for upcoming games.

Table of Contents

Introduction	4
Chapter 1: Web Scraping Data	5
Chapter 2: Elo Rating System	10
Chapter 3: Generating Elo Ratings and Predictions	12
Chapter 4: Optimizing Parameters	19
Chapter 5: Model Evaluation	24
Chapter 6: Model Deployment	31
Conclusion	34
References	35
Appendix	36

Introduction

In NCAA Division I women's ice hockey, team performance is commonly assessed using Pairwise Rankings (PWR), which serve as the primary tool for determining postseason tournament selections. There are currently forty-four teams divided into five conferences, and only eleven qualify for the NCAA tournament. The winners of the five conferences automatically qualify, and the next six teams receive "at large" bids using PWR as the selection criteria. PWR are based on three key components: Rating Percentage Index (RPI), head-to-head comparisons, and record against common opponents. RPI itself incorporates a weighted formula considering a team's winning percentage and strength of schedule (Donnelly, 2023). PWR, though widely used for evaluating teams, have limitations. First, the RPI's static nature and reliance on aggregate record-based statistics may not accurately reflect changes in a team's performance over the season. Additionally, PWR heavily rewards strength of schedule, which can disproportionately favor teams in stronger conferences, potentially overlooking equally competitive teams from smaller or less dominant conferences. Another limitation lies in the binary nature of win/loss outcomes, which ignores other factors such as margin of victory, game location (home ice advantage), or the impact of overtime and shootouts. These factors can provide deeper insights into relative team strength but are not explicitly considered in the PWR structure.

This project aims to explore the use of Elo ratings as a system for ranking Division I women's ice hockey teams, evaluating its ability to model team performance and predict season outcomes. Originally developed for ranking chess players, the Elo rating system is a dynamic system that uses differences in competitors' ratings to predict match outcomes, and subsequently adjust ratings based on the actual outcome while considering factors such as margin of victory and home advantage (Mittal, 2020).

Chapter 1: Web Scraping Data

The goal of this section is to web scrape data from past seasons' schedules into a clean, formatted table. As shown below, the table consists of seven columns: game date, visiting team name, visiting team score, home team name, home team score, whether the game went to overtime, and the type of game (which conference or non-conference). Each row represents a single non-exhibition game played in a given season.

	Date	Visitor	Visitor_Score	Home	Home_Score	OT	Division
1	2023-09-23	Merrimack	0	Clarkson	5	0	Non-Conference
2	2023-09-23	Union	1	St. Cloud State	6	0	Non-Conference
3	2023-09-23	Penn State	0	Northeastern	1	0	Non-Conference
4	2023-09-23	Post	0	Mercyhurst	5	0	Non-Conference
5	2023-09-23	RIT	1	Rensselaer	1	1	Non-Conference
6	2023-09-23	Bemidji State	2	Lindenwood	4	0	Non-Conference
7	2023-09-24	Union	0	St. Cloud State	4	0	Non-Conference
8	2023-09-24	Post	0	Mercyhurst	2	0	Non-Conference
9	2023-09-24	Penn State	2	Northeastern	1	0	Non-Conference
10	2023-09-24	Merrimack	0	Clarkson	3	0	Non-Conference
11	2023-09-24	RIT	1	Rensselaer	1	1	Non-Conference
12	2023-09-24	Bemidji State	1	Lindenwood	4	0	Non-Conference
13	2023-09-28	Wisconsin	6	Lindenwood	0	0	Non-Conference
14	2023-09-29	Franklin Pierce	2	St. Thomas	7	0	Non-Conference
15	2023-09-29	Syracuse	2	Post	0	0	Non-Conference

Figure 1.1: Format of web scraped schedule after cleaning

The schedule and results data are web scraped from *collegehockeynews.com*. This website has the full schedule and results for all games played dating back to the 2017/2018 season, up to and including the schedule for the current 2024/2025 season. This section shows the process of how the data is web scraped and cleaned into a useable format.

A single page of schedule/results on *collegehockeynews.com* shows the game results for an eight-day period, starting at a given date. Figure 1.2 shows the page for the week of September 23, 2023.

Women's Schedule/Results: Week of Sep. 23, 2023

Live Scoreboard Full Schedule Full Standings Stats/Leaders Pairwise

Set game times relative to your time zone: ET | CT | MT | PT | AT (set below as default view)

Saturday, September 23, 2023

Non-Conference	Score	Location	Score	Status
Merrimack	0	at Clarkson	5	Box
Union	1	at St. Cloud State	6	Box
Penn State	0	at Northeastern	1	Box
Post	0	at Mercyhurst	5	Box
RIT	1	at Rensselaer	1	ot Box
Bemidji State	2	at Lindenwood	4	Box
Exhibition	Score	Location	Score	Status
Stonehill	0	at Boston College	7	Box
Durham West	0	at Minnesota	6	Box
New Brunswick	0	at Maine	3	Box
Colgate	4	at St. Lawrence	1	Box
Connecticut	3	at Quinnipiac	4	ot Box

Sunday, September 24, 2023

Non-Conference	Score	Location	Score	Status
Union	0	at St. Cloud State	4	Box
Post	0	at Mercyhurst	2	Box
Penn State	2	at Northeastern	1	Box
Merrimack	0	at Clarkson	3	Box
RIT	1	at Rensselaer	1	ot Box
Bemidji State	1	at Lindenwood	4	Box
Exhibition	Score	Location	Score	Status

Figure 1.2: The *collegehockeynews.com* schedule and results page for the week of 09/23/2023

The *rvest* package is used to scrape the schedule and results into R. Inputting the URL for a website into the *read_html* function from the *rvest* package allows R to navigate to the correct page and scrape the data into tables. The URL for a given week of games is the same for each week, containing the date at the end. It covers eight days' worth of games (ex. Friday to Friday). This makes it simple to trim and append the correct date as the weeks change. For example, the URL for the full schedule and results for the week of October 4, 2024 is <https://www.collegehockeynews.com/women/schedule.php?date=20241004>.

The *scrape_results_function* function web scrapes the results for one of these eight-day periods into an R table. Below is the code for the *scrape_results_function* function.

```

scrape_results_function <- function(date) {
  date_code = gsub("-", "", date)
  url <- paste("https://www.collegehockeynews.com/women/schedule.php?date=",
date_code, sep = "")
  h <- read_html(url)
  tab <- h %>% html_nodes("table")
  obj <- tab %>% html_table()
  clean_sched <- clean_sched_function(obj[[1]])

  return(clean_sched)
}

```

The table is poorly formatted upon the initial web scrape, creating the data frame shown in Figure 1.3.

X1	X2	X3	X4
1 Set game times relative to your time zone: ET CT M...	Set game times relative to your time zone: ET CT M...	Set game times relative to your time zone: ET CT M...	Set game times relative to your time zone: ET CT M...
2			
3 Saturday, September 23, 2023	Saturday, September 23, 2023	Saturday, September 23, 2023	Saturday, September 23, 2023
4 Non-Conference	Non-Conference	Non-Conference	Non-Conference
5 Merrimack	0	at	Clarkson
6 Union	1	at	St. Cloud State
7 Penn State	0	at	Northeastern
8 Post	0	at	Mercyhurst
9 RIT	1	at	Rensselaer
10 Bemidji State	2	at	Lindenwood
11 Exhibition	Exhibition	Exhibition	Exhibition
12 Durham West	0	at	Minnesota
13 Stonehill	0	at	Boston College
14 New Brunswick	0	at	Maine
15 Colgate	4	at	St. Lawrence
16 Connecticut	3	at	Quinnipiac

Figure 1.3: Sample of the initial result of web scraping from *collegehockeynews.com* prior to cleaning

This table is ninety-nine columns wide, but it is only the first six columns that are needed. The headings (dates) and other descriptors (game type) require their own columns, so each row can represent a single game that took place on a given day. The *clean_sched_function* function within *scrape_results_function* is a function that tidies the table into the format seen in Figure 1.1.

Below is the code for *clean_sched_function*.

```

clean_sched_function <- function(date_table) {
  date_table %>%
  select(1:6) %>%
  select(-X3) %>%
  rename(Visitor = X1) %>%
  rename(Visitor_Score = X2) %>%
  rename(Home = X4) %>%
}

```

```

rename(Home_Score = X5) %>%
rename(OT = X6) %>%
mutate(Date_string = ifelse(grepl("[A-Za-z]+", [A-Za-z]+ \\d{1,2}, \\d{4}",
Visitor), Visitor, NA)) %>%
fill(Date_string, .direction = "down") %>%
mutate(Date_string = ifelse(Visitor == "" & Visitor_Score == "", NA,
Date_string)) %>%
filter(!is.na(Date_string)) %>%
mutate(Date = as.Date(Date_string, format = "%A, %B %d, %Y")) %>%
mutate(Division = ifelse(Visitor == Visitor_Score & Visitor != "",
Visitor, NA)) %>%
mutate(Home_Score = as.integer(Home_Score), Visitor_Score =
as.integer(Visitor_Score)) %>%
fill(Division, .direction = "down") %>%
filter(!if_all(everything(), is.na)) %>%
filter(Visitor != Visitor_Score) %>%
select(-c(Date_string)) %>%
mutate(OT = if_else(OT == "", 0, 1)) %>%
relocate(Date, .before = Visitor) %>%
filter(Division != "Exhibition")
}

```

In order to generate the dates that need to be appended to the URL, use the *seq.Date* function. This function creates a list of dates, starting at a given date and including an end date, skipping by a given interval. Due to the formatting of how the complete schedule is put together, the start date is eight days after the date of the first game, and the end date is the date of the last game of the season. The code below shows an example of how a list of dates is generated for the 2023/2024 season and its output.

```

dates_2324 <- seq.Date(from = as.Date("2023-10-01"), to = as.Date("2024-04-
10"), by = "8 days")

```

```

[1] "2023-10-01" "2023-10-09" "2023-10-17" "2023-10-25" "2023-11-02" "2023-11-10"
[7] "2023-11-18" "2023-11-26" "2023-12-04" "2023-12-12" "2023-12-20" "2023-12-28"
[13] "2024-01-05" "2024-01-13" "2024-01-21" "2024-01-29" "2024-02-06" "2024-02-14"
[19] "2024-02-22" "2024-03-01" "2024-03-09" "2024-03-17" "2024-03-25" "2024-04-02"
[25] "2024-04-10"

```

To create the complete schedule for a single season, the results of the first week are scraped, and the rest of the weeks are appended to this initial table. Below is the code for creating this table for the 2023/2024 season.

```
sched_2324 <- scrape_results_function("2023-09-23")
for (week in dates_2324) {
  temp <- scrape_results_function(as.Date(week))
  sched_2324 <- rbind(sched_2324, temp)
}
```

The first line generates the initial table, and then a for loop iterates through the remaining weeks from the `dates_2324` list and appends the results to the schedule, creating the table in Figure 1.1.

Chapter 2: Elo Rating System

The Elo rating system was initially designed to rank chess players and is currently accepted as the international ranking standard for the game, but can also be applied to other one-on-one games and sports (Mittal, 2020). The main idea behind the Elo system is that a player's (or team's) rating reflects their relative strength and is updated after every game depending on the outcome.

Each team starts with an initial rating, typically 1500 for ice hockey, and the rating is adjusted after each game. The amount by which a team's rating changes depends on both the expected outcome and the actual outcome. Using their current ratings (R), the expected win probability (E) is calculated for each team. The formula reflects that the greater the difference in ratings, the more polarized the expected win probabilities will be on the 0-1 scale. The formula below represents the expected win probability for team A when team A plays team B.

$$E_{A|B} = \frac{1}{1 + 10^{\frac{R_B - R_A}{400}}}$$

This formula is then adjusted to account for home ice advantage (H), as teams tend to have a higher probability of winning in their own arena. The adjusted formula below represents the expected win probability for home team A when they play against visiting team B.

$$E_{A|B} = \frac{1}{1 + 10^{\frac{R_B - (R_A + H)}{400}}}$$

After a game, the actual outcome (S) is recorded for a team as a score of 1 for a win, 0 for a loss, or 0.5 for a tie. The expected win probability and actual outcome are then used to calculate updated Elo ratings for each team. The magnitude of the amount by which a team's score changes depends on the initial difference in their ratings. A greater difference (one team highly

avored) results in a smaller change, while a smaller difference results in greater changes. The formula also accounts for a scaling factor (K) that essentially sets the amount of points each game is “worth”. The scaling factor is adjusted by a function of the absolute value of the score differential to reward greater margins of victory (see Chapter 4). Below is the formula for updating a team’s rating.

$$R_{new} = R_{old} + K * (S - E)$$

The new rating is then carried over into a team’s next game where it will be used to calculate new E values. For example, if team A has an Elo rating of 1650 before a game ($R_{old} = 1650$), has a 0.80 probability of winning ($E = 0.80$), and then wins ($S = 1$), their rating will update as shown below if we assume a value of 90 for K:

$$R_{new} = 1650 + 90 * (1 - 0.80)$$

$$R_{new} = 1650 + 90 * 0.20$$

$$R_{new} = 1650 + 18$$

$$R_{new} = 1668$$

Given the magnitude of Elo ratings, an increase of 18 points is not very large. But team A had a very high probability of winning, so they aren’t heavily rewarded for winning a game they were very much favored to win. The Elo system accounts for strength of each opponent when adjusting ratings.

Chapter 3: Generating Elo Ratings and Predictions

In R, the `elo` package is used to compute ratings (Heinzen, 2023). Given a schedule/results data set, the `elo.run` function from the `elo` package updates each team's Elo rating based on the results of each game and internally stores these ratings for use in subsequent games. In this project, we create two functions, one that manually calculates probabilities, and another that updates Elo ratings after games occur.

Predictions

In an Elo model, the primary focus is on predicting the win and loss probabilities for a given pair of teams. However, in sports such as ice hockey where ties are possible, the standard Elo framework doesn't provide the probability for a tie, it only combines it with the win probability. We use logistic regression to predict the probability of a tie, given the absolute value of the difference of the teams' Elo ratings.

The schedule/results table is altered to fit the logistic regression for ties, now including a column whose values are limited to 0 (no tie) or 1 (tie) based on the final scores, and a column that includes the absolute values of the difference in Elo ratings of the teams prior to the game. For the 2023/2024 season, this dataset is called `tie_2324`. For future seasons that are in progress, data from previous seasons will continue to be used, but the 2023/2024 season is used here as the season has been completed. Shown below is the code for creating a logistic regression model that uses Elo rating difference to predict the probability of a tie, as well as its summary output.

```
tie_reg <- glm(Tie ~ Elo_Diff, data = tie_2324, family = binomial)
summary(tie_reg)
```

```

Call:
glm(formula = Tie ~ Elo_Diff, family = binomial, data = tie_2324)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -2.1853800  0.2333686  -9.364  <2e-16 ***
Elo_Diff    -0.0022727  0.0009239  -2.460   0.0139 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 373.29  on 793  degrees of freedom
Residual deviance: 366.17  on 792  degrees of freedom
AIC: 370.17

Number of Fisher Scoring iterations: 6

```

The code below is an example of using the model above to predict tie probability given an Elo rating difference.

```

predict(tie_reg, newdata = data.frame(Elo_Diff = elo_diff), type =
"response")

```

The predict function uses the following formula, where β_0 is the intercept and β_1 is the Elo rating difference predictor coefficient.

$$\begin{aligned}
 P(Tie) &= \frac{1}{1 + e^{-(\beta_0 + \beta_1 * Elo_Diff)}} \\
 &= \frac{1}{1 + e^{-(-2.18538 - 0.0022727 * Elo_Diff)}}
 \end{aligned}$$

When converted to base 10, this formula looks very similar to the original one for calculating the probability of a win:

$$\begin{aligned}
 P(Tie) &= \frac{1}{1 + 10^{-0.4343(\beta_0 + \beta_1 * Elo_Diff)}} \\
 E_{A|B} &= \frac{1}{1 + 10^{\frac{R_B - R_A}{400}}} = \frac{1}{1 + 10^{(0.025 * Elo_Diff)}}
 \end{aligned}$$

Once the tie probability is calculated, it must be incorporated into the win and loss probabilities calculated using the normal formula. To do this, half of the tie probability is subtracted from each team's win and loss probabilities. This approach ensures the total probability sums to 1, while distributing the impact of ties symmetrically across the two primary outcomes. This model is used to create a function that given Elo ratings for the home and visiting teams, outputs the probability of a home win, a visitor win (home loss), and a tie. Below is the code for this function, an example of how to call it, and its output.

```
logistic_elo_probs <- function(home_elo, visitor_elo) {

  # Adjust Elo for home advantage
  home_adjusted_elo <- home_elo + 40
  elo_diff = visitor_elo - home_adjusted_elo

  # Calculate probabilities
  prob_home_win <- 1 / (1 + 10^((elo_diff) / 400))
  prob_visitor_win <- 1 - prob_home_win
  tie_prob <- predict(tie_reg, newdata = data.frame(Elo_Diff =
abs(elo_diff)), type = "response")

  # Return probabilities
  list(
    home_win_prob = prob_home_win - (tie_prob/2),
    visitor_win_prob = prob_visitor_win - (tie_prob/2),
    tie_prob = tie_prob
  )
}
```

```
home_elo <- 1904
visitor_elo <- 1800
logistic_elo_probs(home_elo, visitor_elo)
```

```
$home_win_prob
      1
0.6601001

$visitor_win_prob
      1
0.2678425

$tie_prob
      1
0.07205738
```

Updating Ratings

The data used in this project is from a completed season, which allows us to calculate probabilities and update ratings simultaneously. The Elo rating of each team prior to their first game of the season is used to calculate the initial probabilities. After this first game occurs, ratings must be adjusted based on the actual results and stored for use in later games. The *elo_rating_update* function uses the *logistic_elo_probs* function in order to calculate predictions and then uses these predictions to update Elo ratings accordingly. Below is the code for this function and how it is called.

```
elo_rating_update <- function(schedule, elo_ratings, KValue = 90, HomeAdv =
40) {

  # Copy Elo ratings so they can be updated
  current_elo <- elo_ratings

  # Add new columns to the schedule for results
  schedule$Home_Elo_Before <- NA
  schedule$Visitor_Elo_Before <- NA
  schedule$Home_Elo_After <- NA
  schedule$Visitor_Elo_After <- NA
  schedule$home_win_prob <- NA
  schedule$visitor_win_prob <- NA
  schedule$tie_prob <- NA

  # Loop through each game in the schedule
  for (i in 1:nrow(schedule)) {

    # Get team names and scores
    home_team <- schedule$Home[i]
    visitor_team <- schedule$Visitor[i]
    home_score <- schedule$Home_Score[i]
    visitor_score <- schedule$Visitor_Score[i]

    # Get current Elo ratings for both teams (before the game)
    home_elo <- current_elo[home_team]
    visitor_elo <- current_elo[visitor_team]

    # Store Elo ratings from before the game
    schedule$Home_Elo_Before[i] <- home_elo
    schedule$Visitor_Elo_Before[i] <- visitor_elo

    # Calculate probabilities
    home_adjusted_elo <- home_elo + HomeAdv
    prob_home_win <- logistic_elo_probs(home_adjusted_elo,
    visitor_elo)$home_win_prob
```

```

    prob_visitor_win <- logistic_elo_probs(home_adjusted_elo,
visitor_elo)$visitor_win_prob
    prob_tie <- logistic_elo_probs(home_adjusted_elo, visitor_elo)$tie_prob

    # Determine actual outcome for Elo adjustment
    outcome_home <- ifelse(home_score > visitor_score, 1,
                           ifelse(home_score < visitor_score, 0, 0.5))
    outcome_visitor <- 1 - outcome_home

    # Adjust K value based on score differential (don't adjust if a tie)
    score_diff <- abs(home_score - visitor_score)
    if (home_score != visitor_score) {
      scaled_k <- KValue*(0.6686 * log(score_diff) + 0.8048)
    } else {
      scaled_k <- KValue
    }

    # Update Elo ratings
    current_elo[home_team] <- home_elo + scaled_k * (outcome_home -
prob_home_win)
    current_elo[visitor_team] <- visitor_elo + scaled_k * (outcome_visitor -
prob_visitor_win)

    # Store updated probabilities and ratings after the game
    schedule$home_win_prob[i] <- prob_home_win
    schedule$visitor_win_prob[i] <- prob_visitor_win
    schedule$tie_prob[i] <- prob_tie
    schedule$Home_Elo_After[i] <- current_elo[home_team]
    schedule$Visitor_Elo_After[i] <- current_elo[visitor_team]
  }

  # Return the updated schedule and final Elo ratings
  list(
    updated_schedule = schedule,
    final_elo_ratings = current_elo
  )
}

# Call the function and filter out unnecessary columns
y <- elo_rating_update(sched_2324, scaled_initial2223)[1]
sched_2324_eloDiff <- y$updated_schedule %>% select(-c(1, "Division"))

```

This outputs a complete data frame, shown below, which is used to support the Shiny app (see Chapter 6).

	Date	Visitor	Visitor_Score	Home	Home_Score	OT	Home_Elo_Before	Visitor_Elo_Before	Home_Elo_After	Visitor_Elo_After	home_win_prob	visitor_win_prob	tie_prob
1	2023-09-23	Merrimack	0	Clarkson	5	0	1660.545	1405.155	1686.066	1387.788	0.84923805	0.102593125	0.04816883
2	2023-09-23	Union	1	St. Cloud State	6	0	1557.736	1326.876	1586.262	1306.941	0.83148630	0.117765529	0.05074817
3	2023-09-23	Penn State	0	Northeastern	1	0	1810.463	1635.945	1826.128	1624.421	0.78372702	0.159097347	0.05717563
4	2023-09-23	Post	0	Mercyhurst	5	0	1512.816	1224.977	1534.803	1210.599	0.87011392	0.084938462	0.04494761
5	2023-09-23	RIT	1	Rensselaer	1	1	1398.942	1226.726	1373.602	1257.237	0.78155968	0.160986494	0.05745383
6	2023-09-23	Bemidji State	2	Lindenwood	4	0	1295.385	1299.589	1344.933	1259.509	0.56590708	0.351143076	0.08294985
7	2023-09-24	Union	0	St. Cloud State	4	0	1586.262	1306.941	1607.316	1293.021	0.86490906	0.089318187	0.04577276
8	2023-09-24	Post	0	Mercyhurst	2	0	1534.803	1210.599	1547.326	1202.821	0.89028032	0.068137929	0.04158175
9	2023-09-24	Penn State	2	Northeastern	1	0	1826.128	1624.421	1767.601	1686.858	0.80802844	0.137987859	0.05398370
10	2023-09-24	Merrimack	0	Clarkson	3	0	1686.066	1387.788	1703.212	1376.732	0.87623657	0.079807711	0.04395572
11	2023-09-24	RIT	1	Rensselaer	1	1	1373.602	1257.237	1353.477	1283.176	0.72360324	0.211790413	0.06460634
12	2023-09-24	Bemidji State	1	Lindenwood	4	0	1344.933	1259.509	1388.281	1225.709	0.68711086	0.243971613	0.06891753
13	2023-09-28	Wisconsin	6	Lindenwood	0	0	1388.281	1828.272	1372.245	1852.547	0.08896732	0.865325342	0.04570734
14	2023-09-29	Franklin Pierce	2	St. Thomas	7	0	1354.716	1478.233	1457.401	1390.548	0.39339387	0.517994940	0.08861119

Figure 3.1: Sample of the 2023/2024 schedule with predictions and Elo rating updates

In the future, for seasons in progress probabilities will be calculated using the team’s new rating from the previous game, and ratings will be updated once scores are entered. The *elo_rating_update* and *logistic_elo_probs* functions will be separated and no longer run simultaneously to account for scores not being available when predictions are being made.

Scaling

One of the parameters of the *elo_rating_update* function, *elo_ratings*, represents a vector of Elo ratings that is used as a starting point for the season prior to each team’s first game. If each team were to start with equal Elo ratings of 1500 (the “average” rating), the ratings at the start of the season would emphasize these early games. Not starting at their final rating from the previous season either accounts for movement within teams, as players graduate and sometimes transfer schools. Weaker teams may get better, and stronger teams may not be as good, so closing this gap between top and bottom teams also leaves more room for teams to advance up the rankings during the new season. The vector of initial ratings is the final ratings from the prior season, scaled towards 1500, the rating of an “average” team (Best, 2023). Ratings are scaled towards 1500 using the formula below.

$$R_{scaled} = 1500 + (R - 1500) * 0.7$$

An example calculation for a team that ended the season with a rating of 1850:

$$R_{scaled} = 1500 + (1850 - 1500) * 0.7$$

$$R_{scaled} = 1500 + 350 * 0.7$$

$$R_{scaled} = 1745$$

The vector is created by running the same *elo_rating_update* function on the 2022/2023 schedule, with preliminary Elo ratings set at 1500 for all teams. The ratings from the end of the season are then scaled using the formula above, creating the vector of ratings that are used to initialize the 2023/2024 season prior to its start.

Chapter 4: Optimizing Parameters

In order to make the model as accurate as possible, we want to optimize the scaling factor (K), the home ice advantage parameter (H), and the margin of victory multiplier.

Scaling Factor (K)

The function for optimizing K uses a schedule and results in order to calculate Elo-based expected win probabilities. It then loops through the different K values we are considering, and calculates the mean squared error (MSE) for each one. The code uses R's *elo.run* function to calculate probabilities and update ratings, as this then allows us to use the *mse* function. In this case, the error is between the predicted win probability for the home team and the actual outcome for that team. The error is first calculated and squared for an individual game by squaring the difference between the win probability (E) and actual outcome (S), then this amount is summed over all games and divided by the total number of games in that season (n).

$$MSE = \frac{\sum_{i=1}^n (S_i - E_i)^2}{n}$$

This process occurs for each value of K. After the MSE for each K value is calculated, the K value with the lowest MSE is selected, and a plot is created to visualize this value. Below is the function and example code for creating the plot for the 2023/2024 season, adapted from Hope Donoghue's thesis (Donoghue, 2024).

```
calculate_k <- function(data) {
  # Number of K values to try
  ndots <- 140

  # Empty vector that will store MSEs
  elo_MSE <- numeric(ndots)

  # K values to try
```

```

kk <- seq(50, 189, by = 1)

# Loop to compute MSE for each k
for (dot in 1:ndots) {
  elo_d1 <- elo.run(score(Home_Score, Visitor_Score) ~ Home + Visitor,
data = data, k = kk[dot])
  elo_MSE[dot] <- mse(elo_d1)
}

# Find the k value where MSE is the lowest
optimal_index <- which.min(elo_MSE)
optimal_k <- kk[optimal_index]
optimal_MSE <- elo_MSE[optimal_index]

# Print the results
cat("Optimal k value:", optimal_k, "\n")
cat("Minimum MSE:", optimal_MSE, "\n")

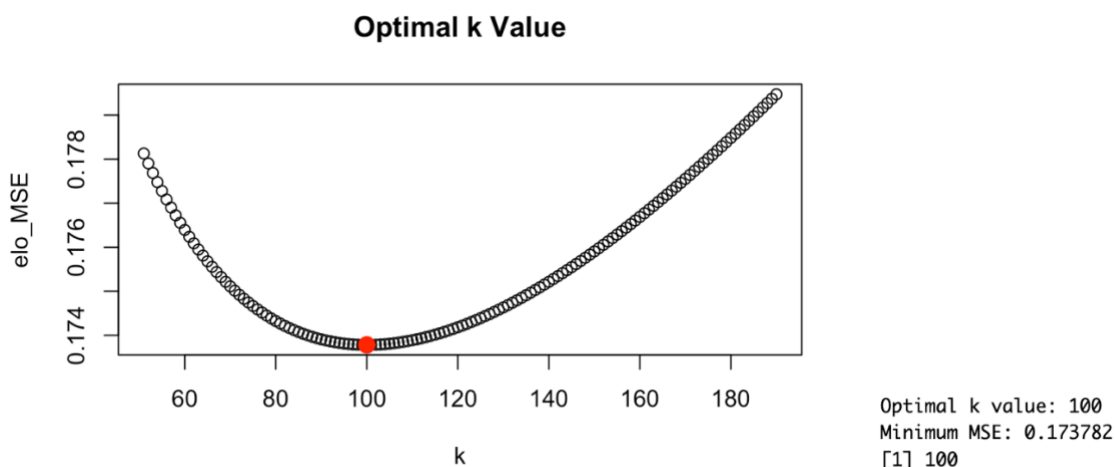
# Plot the graph
plot(elo_MSE ~ kk, xlab = "k", main = "Optimal k Value")

# Highlight the optimal point
points(optimal_k, optimal_MSE, col = "red", pch = 19, cex = 1.5)

return(optimal_k)
}

# Call the function
calculate_k(sched_2324)

```



This output shows that the K value with the lowest MSE for the 2023/2024 season is 100. The optimal K value is taken for five seasons of scraped data, shown in the table below. Note that the 2020/2021 season is omitted, as it was shortened due to COVID-19.

Season	2018/2019	2019/2020	2021/2022	2022/2023	2023/2024
Optimal K Value	70	73	102	92	100

An average weighted by number of games is taken, resulting in a K value of 87.9, rounded up to 90 to reflect the more recent above average values.

Home Ice Advantage (H)

The function for optimizing home ice advantage works very similarly to the one used for the scaling factor. Like the process for optimizing K, it loops through different H values and calculates the mean squared error (MSE) for each one. Again, the error is between the predicted win probability for the home team and the actual outcome for that team. This process occurs for each value of H. After the MSE for each H value is calculated, the H value with the lowest MSE is selected, and a plot is created to visualize this value. Below is the function and example code for creating the plot for the 2023/2024 season, using our optimized K value of 90.

```
calculate_home_advantage <- function(data, k = 90) {

  # Number of H values to try (0 to 70)
  ndots <- 71

  # Home advantage values
  hh <- seq(0, 70, by = 1)

  # Empty vector to store MSE results
  elo_MSE <- numeric(ndots)

  # Loop to calculate MSE for each H
  for (dot in 1:ndots) {
    elo_d1 <- elo.run(score(Home_Score, Visitor_Score) ~ adjust(Home,
hh[dot]) + Visitor, data = data, k = k)
    elo_MSE[dot] <- mse(elo_d1)
  }

  # Find the H value with the lowest MSE
  optimal_index <- which.min(elo_MSE)
  optimal_H <- hh[optimal_index]
  optimal_MSE <- elo_MSE[optimal_index]
```

```

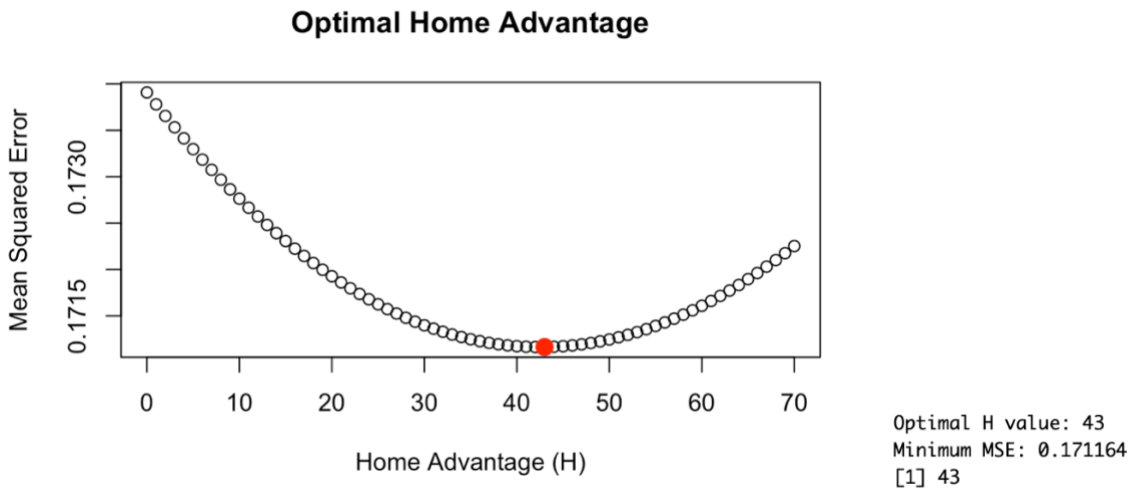
# Print results
cat("Optimal H value:", optimal_H, "\n")
cat("Minimum MSE:", optimal_MSE, "\n")

# Plot the results
plot(elo_MSE ~ hh, xlab = "Home Advantage (H)", ylab = "Mean Squared
Error",
     main = "Optimal Home Advantage")
points(optimal_H, optimal_MSE, col = "red", pch = 19, cex = 1.5)

return(optimal_H)
}

calculate_home_advantage(sched_2324)

```



This output shows that the H value with the lowest MSE for the 2023/2024 season is 43. The optimal K value is taken for five seasons of scraped data, shown in the table below.

Season	2018/2019	2019/2020	2021/2022	2022/2023	2023/2024
Optimal H Value	66	32	39	29	43

An average weighted by number of games is taken, resulting in an H value of 41.5, rounded down to 40 to reflect the more recent lower values.

Margin of Victory

The margin of victory multiplier adjusts K to account for whether the game was close or a runaway, rewarding the latter. FiveThirtyEight is an American polling analysis company, and

they use Elo ratings to make predictions pertaining to the National Hockey League (NHL). This project uses their margin of victory multiplier, which is shown below.

$$\text{Multiplier} = 0.6686 \times \ln(\text{Margin of Victory}) + 0.8048$$

where Margin of Victory is the absolute value of the goal differential

For example, in a game that is won 4-2:

$$\text{Multiplier} = 0.6686 \times \ln(|4 - 2|) + 0.8048$$

$$\text{Multiplier} = 0.6686 \times \ln(2) + 0.8048$$

$$\text{Multiplier} = 1.27$$

Given $K = 90$:

$$K_{adj} = 90 \times 1.27$$

$$K_{adj} = 114.14$$

This adjusted K value is plugged into the Elo rating update formula, which functions as usual:

$$R_{new} = R_{old} + K_{adj} * (S - E)$$

In the function for updating ratings using predictions and game results (*elo_rating_update*), the K value is only adjusted for games that do not end in a tie. This makes sense both logically and mathematically. When two teams tie, neither team should be further rewarded or punished, so the K value is not adjusted. This is reflected mathematically as when the game ends in a tie the goal differential is zero, and the natural log of zero is undefined, so instead of multiplying K by 0.8048 which would decrease K, we choose not to adjust K at all.

Chapter 5: Model Evaluation

Poisson Model

In order to assess the accuracy of the Elo model, a Poisson distribution is used. A Poisson distribution models the probability of a certain number of events occurring within a given fixed time period or number of trials. The only parameter is λ , which represents the average number of events that occur. In this context, λ values are the offensive and defensive ratings for a given team, which represent a team's goal scoring and allowance rates, respectively. In order to use these ratings to evaluate the Elo model, we followed Hope Donoghue's thesis paper that used Bessel functions in the context of sports to calculate probabilities (Donoghue, 2024). Bessel functions are a series of solutions to a specific differential equation (Niedziela, 2008). The probability of a tie is calculate using the following formula:

$$P(Tie) = J_0(2\sqrt{\lambda_1\lambda_2})e^{-(\lambda_1\lambda_2)}$$

where λ_1 and λ_2 represent offensive ratings for two teams, and $J_0(x)$ is a zero order Bessel function

The k^{th} order Bessel function below gives the probability of a team winning by k goals:

$$P(\text{Team 1 wins by } k \text{ goals}) = J_k(2\sqrt{\lambda_1\lambda_2})\left(\frac{\lambda_1}{\lambda_2}\right)^{k/2}e^{-(\lambda_1\lambda_2)}$$

To find the overall probability of team 1 winning, sum the k^{th} order Bessel functions over all possible margins of victory (k):

$$P(\text{Team 1 wins}) = \sum_{k=1}^{\infty} J_k(2\sqrt{\lambda_1\lambda_2})\left(\frac{\lambda_1}{\lambda_2}\right)^{k/2}e^{-(\lambda_1\lambda_2)}$$

Given the typical margin of victory in an ice hockey game, we can confidently account for most of the probability stopping at $k = 20$:

$$P(\text{Team 1 wins}) = \sum_{k=1}^{20} J_k(2\sqrt{\lambda_1\lambda_2}) \left(\frac{\lambda_1}{\lambda_2}\right)^{k/2} e^{-(\lambda_1\lambda_2)}$$

The probability of a loss is the remaining probability outside of a win or tie:

$$P(\text{Team 1 loses}) = 1 - P(\text{Tie}) - P(\text{Team 1 wins})$$

The Elo model is tested using results from the 2018/2019 season, as the offensive and defensive ratings are available via Lock’s CHODR (Lock, 2019). The final offensive and defensive ratings for the 2018/2019 season from CHODR gives the number of goals a team is expected to score and allow against the “average” team, shown in Figure 5.1.

	Team	Off_Rate	Def_Rate
1	Wisconsin	3.97	0.88
2	Minnesota	4.62	1.50
3	Northeastern	3.48	1.82
4	Clarkson	3.38	1.76
5	Boston College	3.40	1.85
6	Princeton	3.45	2.11
7	Ohio State	3.19	1.94
8	Boston University	3.18	1.94
9	Cornell	2.89	1.82
10	Minnesota Duluth	3.34	2.22
11	Providence	2.81	1.86
12	Merrimack	2.74	2.10

Figure 5.1: Sample of final offensive and defensive ratings from the 2018/2019 season

This allows you to simply use a single team’s offensive and defensive ratings as λ_1 and λ_2 respectively, as the defensive rating (from against an average team) can be treated as said average team’s offensive rating. The *bessell* function from the Bessel package in R is used to calculate the probability of a win, loss, and tie against the “average” team (Donoghue, 2024). The function below takes the two λ values as parameters and returns a list of the three probabilities.

```
poisson_probs <- function(lambda1, lambda2) {
  prob_tie <- bessell(2*sqrt(lambda1*lambda2), 0)*exp(-(lambda1+lambda2))
```

```

# Sum of win probabilities over all margins of victory
total <- 0

# Loop through the k = 20 margins of victory
for(i in 1:20){
  prob_win_iteration <- bessell(2*sqrt(lambda1*lambda2), i)*
(lambda1/lambda2)^(i/2) *exp(-(lambda1+lambda2))

  total = total + prob_win_iteration
  i=i+1
}

# Win and loss probabilities
prob_win <- total
prob_loss <- 1 - (prob_tie) - (prob_win)

return(list(prob_win, prob_tie, prob_loss))
}

```

We loop through a list of all teams with their offensive and defensive ratings (`off_def_1819`) using the *poisson_probs* function. This creates a data frame of teams and their three probabilities for a game against an “average” team. The code below shows this function and its output.

```

prob_rate <- data.frame(prob_win = numeric(), prob_tie = numeric(),
prob_loss = numeric())

# Loop through all 35 teams
for (i in c(1:35)) {
  temp <- poisson_probs(off_def_1819$Off_Rate[i], off_def_1819$Def_Rate[i])
  prob_rate <- rbind(prob_rate, temp)
}

```

	team	prob_win	prob_tie	prob_loss
1	Wisconsin	0.88999004	0.07067380	0.03933616
2	Minnesota	0.86116397	0.07594033	0.06289570
3	Northeastern	0.69161211	0.14008164	0.16830626
4	Clarkson	0.68794700	0.14311506	0.16893794
5	Boston College	0.67497445	0.14499675	0.18002880
6	Princeton	0.63705738	0.14961027	0.21333235
7	Ohio State	0.62713000	0.15766121	0.21520879
8	Boston University	0.62555100	0.15814008	0.21630892
9	Cornell	0.60019726	0.16994016	0.22986258
10	Minnesota-Duluth	0.60117918	0.15650278	0.24231804
11	Providence	0.57884174	0.17462645	0.24653180
12	Merrimack	0.52173424	0.17983085	0.29843492

Figure 5.2: Sample output of the *poisson_probs* function for the 2018/2019 season

The resulting probabilities are used to calculate the Elo rating for each team at the end of the 2018/2019 season. Since an Elo model only predicts a win or a loss, the output from the Bessel function is modified, adding half of the probability of a tie to the win probability, stored in a new object called `elo_pois_1819`. This new win probability is then used to solve for the Elo rating of a team using the rearranged formula for E below, using the “average” team with a rating of 1500:

$$R = 1500 - 400 * \log \frac{1 - E}{E}$$

We then use the function below to perform this calculation, loop through each team’s win probability, and return the resulting Elo ratings.

```
pois_to_elo <- function(pois_prob) {
  elo <- 1500 - 400*log10((1-pois_prob)/pois_prob)
  return (elo)
}

# Object to store Elo ratings
elo_from_pois <- data.frame(team = character(), elo = numeric(),
  elo_from_pois = numeric())

# Loop through each of the 35 teams and calculate an Elo rating
for (i in c(1:35)) {
  temp <- pois_to_elo(elo_pois_1819$prob_win[i])
  elo_from_pois <- rbind(elo_from_pois, temp)
}
```

The resulting Elo ratings are then appended to a data frame of final ratings for the 2018/2019 season, calculated using our Elo model (`elos_1819`). The difference in the resulting Elo ratings from the Elo model and Poisson model is also calculated to compare them.

```
colnames(elo_from_pois) <- c("elo_from_pois")
elo_from_pois$team <- elo_pois_1819$team

# Join the two Elo data frames by team name, calculate difference
elo_from_pois1819 <- elo_from_pois %>% left_join(elos_1819) %>%
  select(2,3,1) %>% mutate(diff = elo rating - elo from pois)
```

The output below shows the first 12 rows that the code produces, where `elo_rating` represents the Elo rating from the Elo model, `elo_from_pois` represents the Elo rating from the Poisson model, and `diff` represents the difference between them.

	team	elo_rating	elo_from_pois	diff
1	Wisconsin	1934.300	1937.253	-2.952071
2	Minnesota	1862.675	1880.032	-17.357070
3	Northeastern	1792.868	1701.819	91.049322
4	Clarkson	1914.871	1699.769	215.102081
5	Boston College	1777.015	1688.515	88.499972
6	Princeton	1587.946	1657.119	-69.172852
7	Ohio State	1665.134	1652.150	12.984135
8	Boston University	1745.718	1651.030	94.687495
9	Minnesota-Duluth	1582.726	1630.489	-47.763466
10	Cornell	1734.948	1635.087	99.861516
11	Providence	1724.879	1620.012	104.867177
12	Colgate	1639.876	1579.398	60.478255

Figure 5.3: Elo ratings from the Elo model and Poisson model

From the difference column, we can see that there are different levels of variability, both in the positive and negative directions, which is a positive indication of no consistent over or under estimation. With the high magnitude of Elo ratings, it was not expected for the ratings to be near identical. However, despite the differences, some upwards of hundreds of “points” off, the ranking of the teams is almost completely preserved across all thirty-five teams from the 2018/2019 season. While only applicable to the 2018/2019 season, the consistency demonstrates that the ratings from our Elo model are reasonable.

PWR and USCHO Poll

The USCHO poll is an opinion-based poll in which twenty coaches and media members from across the United States vote on who they think the top teams are (College Hockey Inc., 2025). Each ranking is worth a certain number of points. The number one ranking is worth fifteen points, and each lower ranking is worth one point less than the one above it. Every week, each of the voting members ranks their top fifteen teams, and the points each team earns from the

twenty rankings are added up to create the overall rankings. For example, if each of the twenty voting members ranks Wisconsin as the number one team, Wisconsin earns $20 \times 15 = 300$ points, the maximum possible points, putting them in the number one spot. When the weekly poll rankings are compared to the weekly Elo rankings (see Chapter 6), like with the Poisson model some individual rankings vary, but the top ten to twelve teams are consistently made up of the same teams across both methods.

The NCAA's PWR are not released on a weekly basis but are updated upon the completion of a game, so only the final pairwise rankings are preserved for a completed season. Historically, the NCAA's PWR and the USCHO poll present a very similar ranking of teams by the end of the season, as is the case here. At the end of the 2023/2024 season, the PWR again show a very similar ranking to our Elo rating system, with some variation in individual rankings but the same teams comprising the top ten to twelve teams. At the end of the season, based on the rankings, the Elo model has the same teams qualifying for the NCAA tournament. Overall, it appears that the Elo system effectively extends to team sports, the two-team nature mimicking that of two individual chess players. However, there is more variability in team composition between seasons (versus with an individual chess player), so re-evaluating the model as teams change from season to season will be important for future work.

Applications

Our Elo model producing near identical rankings to the NCAA's existing system gives confidence in the viability of Elo as a ranking system for ice hockey. Based on our rankings at the end of the 2023/2024 playoff season (prior to tournament selection), the same teams would receive at large bids. But by producing essentially the same rankings, is this really any better or

different than the existing rankings? The answer may be no. As far as the actual rankings go, this project shows that Elo can be just as effective or as accurate as PWR. But where Elo surpasses PWR is in its ability to make predictions. With probabilities at the root of its ratings, the Elo system has the unique ability to produce overall ratings while also offering extractable predictions. Not only can this just be interesting to see, but it also allows you to quantify an upset (the weaker team winning). A team beating a higher ranked team doesn't always constitute an upset, as they may be only separated by a few points in the standings. But by looking at the probabilities, a threshold for an upset could be quantified. For example, a team winning a game in which they only had a 0.2 probability of winning numerically defines the team as an underdog. This can have interesting applications in sports betting, in areas such as money line betting where payouts are based on which team is predicted to win.

Chapter 6: Model Deployment

Shiny App

A Shiny app displays ratings that are updated and ranked on a weekly basis, as well as predictions and results for the games in a selected week. The link to the app is: <https://tlum.shinyapps.io/WHKYEloRatings/>. When a user opens the app, the first tab that pops up is the Elo ratings ranked in descending order for the most recent week, as shown below.

NCAA Division I Women's Ice Hockey Elo Ratings

Elo Ratings [Schedule](#)

Select Week: 8

Show 20 entries Search:

	Team	Elo Rating	Rank Change
1	Wisconsin	1989	–
2	Ohio State	1984	–
3	Clarkson	1974	+1 ↑
4	Colgate	1898	-1 ↓
5	Minnesota	1820	+1 ↑
6	Cornell	1805	+1 ↑
7	Minnesota-Duluth	1772	+1 ↑
8	St. Lawrence	1767	-3 ↓
9	Quinnipiac	1752	–
10	Connecticut	1711	+1 ↑
11	St. Cloud State	1679	-1 ↓
12	Yale	1674	+1 ↑

Figure 6.1: Elo ratings tab on Shiny app

The user can use the drop-down menu on the left to select a week of the season to view the rankings for. The rightmost column shows a team's rank change relative to the previous week. For example, if a team had the second highest ranking last week, but falls to the fourth highest after a weekend of games, the third column would show a -2 in red.

The app also has a schedule tab, which shows the home team win, visiting team win, and tie probabilities for each game in the selected week. It also shows the actual score of each game.

NCAA Division I Women's Ice Hockey Elo Ratings

Elo Ratings Schedule

Select Week: 8

Show 10 entries

Search:

	Date	Home	Home_Score	Visitor	Visitor_Score	home_win_prob	visitor_win_prob	tie_prob
1	2023-11-08	Clarkson	6	St. Lawrence	3	0.52	0.392	0.088
2	2023-11-08	Quinnipiac	12	Post	1	0.963	0.012	0.025
3	2023-11-09	St. Thomas	4	Minnesota State	1	0.497	0.412	0.091
4	2023-11-10	Maine	0	Connecticut	1	0.182	0.758	0.06
5	2023-11-10	Vermont	3	Boston College	6	0.558	0.358	0.084
6	2023-11-10	Boston University	2	New Hampshire	4	0.663	0.266	0.072
7	2023-11-10	Merrimack	2	Holy Cross	2	0.322	0.599	0.079
8	2023-11-10	Northeastern	6	Providence	1	0.581	0.338	0.081
9	2023-11-10	Sacred Heart	0	Stonehill	1	0.315	0.607	0.078
10	2023-11-10	Long Island	4	Saint Anselm	2	0.715	0.219	0.066

Showing 1 to 10 of 32 entries

Previous 1 2 3 4 Next

Figure 6.2: Schedule tab on Shiny app

For future seasons, an additional drop-down menu will be added, allowing users to look at ratings across each year.

Future Automation

The data used in this project is from the completed 2023/2024 season. The current version of the app pulls from a file called *sched_2324_full.csv* that is stored in a GitHub repository (<https://github.com/taylorlum/D1WHKYeloRatings>). That file contains all of the necessary information for the app, including the schedule, scores, Elo ratings for both teams before a game, Elo ratings for both teams after a game, as well as the win and tie probabilities.

For future seasons, ratings will be updated regularly as the season progresses. Currently, a YAML file called *python_schedule.yml* in the workflow folder in the repository utilizes a feature called GitHub Actions, which allows it to trigger a job on a schedule (GitHub, n.d.). In this case, the job is running the code that scrapes and cleans the schedule/results data. The workflow triggers the *python_schedule_scrape.py* file to run, resulting in an updated schedule/results csv file that overwrites the existing one. The code for both of these files can be found in the

Appendix or the repository linked above. The way the code interacts with *collegehockeynews.com* results in the season's complete schedule being scraped, and the weekly trigger will update the file to include the scores from the previous week. The *python_schedule_scrape.py* file has the same functionality as the original R scraping code, but has just been converted into Python as it seems to allow the workflow to run more smoothly. Additional workflow and Python files will be created to update Elo ratings as games occur (and schedule file is updated), which will subsequently adjust the probability predictions for the unplayed games. A new csv will be saved to the repository having the same structure as *sched_2324_full.csv* file. The Shiny app will then pull from the updated data, resulting in updated rankings and new predictions that reflect these updated ratings.

Conclusion

In this project, we evaluated the Elo rating system as an alternative to the current Pairwise Rankings (PWR) system used by the NCAA for Division I women's ice hockey. We used MSE to optimize the parameters of the Elo model, which was used to make predictions and update teams' ratings. After running the Elo model through the whole 2023/2024 season, we evaluated its performance by comparing its final ratings to those derived from Poisson-based probabilities as well as to the NCAA's PWR and the USCHO opinion-based poll. While there were some differences in individual rankings, we found that the Elo model accurately ranked teams, predicting the same ones in its top ten to twelve slots. We then created a Shiny app to display predictions and rankings, evaluating teams on a weekly basis.

In the future, the Shiny app will update rankings and make predictions as games occur throughout a season in progress, opposed to waiting for the season to end. The prediction and rating update function framework will be modified, and GitHub workflows will automate the process of collecting updated data, making predictions, and updating the ratings. While the Elo rating system proved to be just as effective as PWR, sending the same teams to the NCAA tournament, its strength lies in its predictive, yet simple, nature. We could explore its efficacy at making predictions and rankings in other two-party sports, as well as explore its application in sports betting. Finally, we optimized several parameters in this project, accounting for factors such as margin of victory and home ice advantage. Including other factors in the model that may contribute to the strength of a team may be worth further exploration.

References

- Best, R. (2023, January 13). *How our NHL predictions work*. FiveThirtyEight. <https://fivethirtyeight.com/methodology/how-our-nhl-predictions-work/>
- College Hockey Inc. (2025, March 25). *Women's national polls*. <https://www.collegehockeyinc.com/womens-national-polls/>
- Donnelly, P. (2023, March 21). *Division I men's ice hockey tournament selection process, explained*. NCAA. <https://www.ncaa.com/news/icehockey-men/article/2023-03-13/division-i-mens-ice-hockey-tournament-selection-process-explained>
- Donoghue, H. (2024). *Comparing rating methods in NCAA Division III women's soccer* [Undergraduate thesis, St. Lawrence University].
- GitHub. (n.d.). *GitHub Actions documentation*. <https://docs.github.com/en/actions>
- Heinzen, E. (2023, August 23). *elo: Ranking teams by Elo rating and comparable methods*. R Project. <https://cran.r-project.org/web/packages/elo/index.html>
- Lock, R. (2019, March 24). *WCHODR: Women's college hockey offensive/defensive ratings – final 2018-2019 season*. St. Lawrence University. <https://it.stlawu.edu/~chodr/wchodr/wchodr2019/wch2019.html>
- Mittal, R. (2020, September 11). *What is an ELO rating?* Medium. <https://medium.com/purple-theory/what-is-elo-rating-c4eb7a9061e0>
- Niedziela, J. (2008, October 29). *Bessel functions and their applications*. University of Tennessee, Knoxville. <https://scs.phys.utk.edu/~moreo/mm08/niedzilla.pdf>

Appendix

Web Scraping

```

scrape_results_function <- function(date) {
  date_code = gsub("-", "", date)
  url <- paste("https://www.collegehockeynews.com/women/schedule.php?date=",
date_code, sep = "")
  h <- read_html(url)
  tab <- h %>% html_nodes("table")
  obj <- tab %>% html_table()
  clean_sched <- clean_sched_function(obj[[1]])

  return(clean_sched)
}

clean_sched_function <- function(date_table) {
  date_table %>%
  select(1:6) %>%
  select(-X3) %>%
  rename(Visitor = X1) %>%
  rename(Visitor_Score = X2) %>%
  rename(Home = X4) %>%
  rename(Home_Score = X5) %>%
  rename(OT = X6) %>%
  mutate(Date_string = ifelse(grepl("[A-Za-z]+", [A-Za-z]+ \\d{1,2}, \\d{4}",
Visitor), Visitor, NA)) %>%
  fill(Date_string, .direction = "down") %>%
  mutate(Date_string = ifelse(Visitor == "" & Visitor_Score == "", NA,
Date_string)) %>%
  filter(!is.na(Date_string)) %>%
  mutate(Date = as.Date(Date_string, format = "%A, %B %d, %Y")) %>%
  mutate(Division = ifelse(Visitor == Visitor_Score & Visitor != "", Visitor,
NA)) %>%
  mutate(Home_Score = as.integer(Home_Score), Visitor_Score =
as.integer(Visitor_Score)) %>%
  fill(Division, .direction = "down") %>%
  filter(!if_all(everything(), is.na)) %>%
  filter(Visitor != Visitor_Score) %>%
  select(-c(Date_string)) %>%
  mutate(OT = if_else(OT == "", 0, 1)) %>%
  relocate(Date, .before = Visitor) %>%
  filter(Division != "Exhibition")
}

```

Generating Predictions and Updating Ratings

```

logistic_elo_probs <- function(home_elo, visitor_elo) {

  # Adjust Elo for home advantage
  home_adjusted_elo <- home_elo + 40
  elo_diff = visitor_elo - home_adjusted_elo

  # Calculate probabilities
  prob_home_win <- 1 / (1 + 10^((elo_diff) / 400))
  prob_visitor_win <- 1 - prob_home_win
  tie_prob <- predict(tie_reg, newdata = data.frame(Elo_Diff =
abs(elo_diff)), type = "response")

  # Return probabilities
  list(
    home_win_prob = prob_home_win - (tie_prob/2),
    visitor_win_prob = prob_visitor_win - (tie_prob/2),
    tie_prob = tie_prob
  )
}

elo_rating_update <- function(schedule, elo_ratings, KValue = 90, HomeAdv =
40) {

  # Copy Elo ratings to update dynamically
  current_elo <- elo_ratings

  # Add new columns to the schedule for results
  schedule$Home_Elo_Before <- NA
  schedule$Visitor_Elo_Before <- NA
  schedule$Home_Elo_After <- NA
  schedule$Visitor_Elo_After <- NA
  schedule$home_win_prob <- NA
  schedule$visitor_win_prob <- NA
  schedule$tie_prob <- NA

  # Loop through each game in the schedule
  for (i in 1:nrow(schedule)) {
    # Get team names and scores
    home_team <- schedule$Home[i]
    visitor_team <- schedule$Visitor[i]
    home_score <- schedule$Home_Score[i]
    visitor_score <- schedule$Visitor_Score[i]

    # Get current Elo ratings for both teams (before the game)
    home_elo <- current_elo[home_team]
    visitor_elo <- current_elo[visitor_team]

    # Store Elo ratings from before the game
    schedule$Home_Elo_Before[i] <- home_elo
    schedule$Visitor_Elo_Before[i] <- visitor_elo

    # Calculate probabilities
    home_adjusted_elo <- home_elo + HomeAdv

```

```

    prob_home_win <- logistic_elo_probs(home_adjusted_elo,
    visitor_elo)$home_win_prob
    prob_visitor_win <- logistic_elo_probs(home_adjusted_elo,
    visitor_elo)$visitor_win_prob
    prob_tie <- logistic_elo_probs(home_adjusted_elo, visitor_elo)$tie_prob

    # Determine actual outcome for Elo adjustment
    outcome_home <- ifelse(home_score > visitor_score, 1,
                           ifelse(home_score < visitor_score, 0, 0.5))
    outcome_visitor <- 1 - outcome_home

    # Adjust KValue based on score differential (don't adjust if a tie)
    score_diff <- abs(home_score - visitor_score)
    if (home_score != visitor_score) {
      scaled_k <- KValue*(0.6686 * log(score_diff) + 0.8048)
    } else {
      scaled_k <- KValue
    }

    # Update Elo ratings
    current_elo[home_team] <- home_elo + scaled_k * (outcome_home -
    prob_home_win)
    current_elo[visitor_team] <- visitor_elo + scaled_k * (outcome_visitor -
    prob_visitor_win)

    # Store updated probabilities and ratings after the game
    schedule$home_win_prob[i] <- prob_home_win
    schedule$visitor_win_prob[i] <- prob_visitor_win
    schedule$tie_prob[i] <- prob_tie
    schedule$Home_Elo_After[i] <- current_elo[home_team]
    schedule$Visitor_Elo_After[i] <- current_elo[visitor_team]
  }

  # Return the updated schedule and final Elo ratings
  list(
    updated_schedule = schedule,
    final_elo_ratings = current_elo
  )
}

```

Scaling Factor (K) Optimization

```

calculate_k <- function(data) {

  # Number of K values to try
  ndots <- 140

  # Empty vector that will store MSEs
  elo_MSE <- numeric(ndots)

  # K values to try
  kk <- seq(50, 189, by = 1)

  # Loop to compute MSE for each k
  for (dot in 1:ndots) {

```

```

    elo_d1 <- elo.run(score(Home_Score, Visitor_Score) ~ Home + Visitor, data
= data, k = kk[dot])
    elo_MSE[dot] <- mse(elo_d1)
  }

  # Find the k value where MSE is the lowest
  optimal_index <- which.min(elo_MSE)
  optimal_k <- kk[optimal_index]
  optimal_MSE <- elo_MSE[optimal_index]

  # Print the results
  cat("Optimal k value:", optimal_k, "\n")
  cat("Minimum MSE:", optimal_MSE, "\n")

  # Plot the graph
  plot(elo_MSE ~ kk, xlab = "k", main = "Optimal k Value")

  # Highlight the optimal point
  points(optimal_k, optimal_MSE, col = "red", pch = 19, cex = 1.5)

  return(optimal_k)
}

```

Home Ice Advantage (H) Optimization

```

calculate_home_advantage <- function(data, k = 90) {

  # Number of H values to try (0 to 70)
  ndots <- 71

  # Home advantage values
  hh <- seq(0, 70, by = 1)

  # Empty vector to store MSE results
  elo_MSE <- numeric(ndots)

  # Loop to compute MSE for each H
  for (dot in 1:ndots) {
    elo_d1 <- elo.run(score(Home_Score, Visitor_Score) ~ adjust(Home,
hh[dot]) + Visitor, data = data, k = k)
    elo_MSE[dot] <- mse(elo_d1)
  }

  # Find the H value with the lowest MSE
  optimal_index <- which.min(elo_MSE)
  optimal_H <- hh[optimal_index]
  optimal_MSE <- elo_MSE[optimal_index]

  # Print results
  cat("Optimal H value:", optimal_H, "\n")
  cat("Minimum MSE:", optimal_MSE, "\n")

  # Plot the results
  plot(elo_MSE ~ hh, xlab = "Home Advantage (H)", ylab = "Mean Squared
Error",

```

```

    main = "Optimal Home Advantage")
points(optimal_H, optimal_MSE, col = "red", pch = 19, cex = 1.5)

return(optimal_H)
}

```

Poisson Model Probabilities

```

poisson_probs <- function(lambda1, lambda2){

  prob_tie <- besselI(2*sqrt(lambda1*lambda2), 0)*exp(-(lambda1+lambda2))

  # Sum of win probabilities over all margins of victory
  total <- 0

  # Loop through the k = 20 margins of victory
  for(i in 1:20){
    prob_win_iteration <- besselI(2*sqrt(lambda1*lambda2), i)*
(lambda1/lambda2)^(i/2) *exp(-(lambda1+lambda2))

    total = total + prob_win_iteration
    i=i+1
  }
  prob_win <- total
  prob_loss <- 1 - (prob_tie) - (prob_win)

  return(list(prob_win, prob_tie, prob_loss))
}

```

Calculating Elo Ratings from Poisson Probabilities

```

pois_to_elo <- function(pois_prob) {
  elo <- 1500 - 400*log10((1-pois_prob)/pois_prob)
  return (elo)
}

```

Shiny App

```

# Load libraries
library(rsconnect)
library(shiny)
library(DT)
library(readr)
library(dplyr)
library(lubridate)
library(tidyr)

data_url <-
"https://raw.githubusercontent.com/taylorlum/D1WHKYeloRatings/refs/heads/main
/sched_2324_full.csv"

```

```

# Week 1
start_date <- as.Date("2023-09-18")

# Load the data
load_data <- function() {
  read_csv(data_url, show_col_types = FALSE) %>%
    mutate(
      date = as.Date(Date),
      week = as.integer(floor(as.numeric(difftime(Date, start_date, units =
"days")) / 7)) + 1,
      home_win_prob = round(home_win_prob, 3),
      visitor_win_prob = round(visitor_win_prob, 3),
      tie_prob = round(tie_prob, 3)
    )
}

ui <- fluidPage(

  titlePanel("NCAA Division I Women's Ice Hockey Elo Ratings"),

  tabsetPanel(

    tabPanel("Elo Ratings",
      sidebarLayout(
        sidebarPanel(
          selectInput("week_elo", "Select Week:", choices = NULL)
        ),
        mainPanel(
          DTOutput("elo_table")
        )
      )
    ),
    tabPanel("Schedule",
      sidebarLayout(
        sidebarPanel(
          selectInput("week", "Select Week:", choices = NULL)
        ),
        mainPanel(
          DTOutput("schedule_table")
        )
      )
    )
  )

)

server <- function(input, output, session) {

  full_data <- reactive({
    load_data()
  })

  observe({
    week_choices <- sort(unique(full_data()$week))
    updateSelectInput(session, "week", choices = week_choices, selected =
max(week_choices))
  })
}

```

```

    updateSelectInput(session, "week_elo", choices = week_choices, selected =
max(week_choices))
  })

# The columns needed for the schedule tab
filtered_data <- reactive({
  req(input$week)
  full_data() %>% filter(week == input$week) %>%
    select(c(2, 5, 6, 3, 4, 12:14))
})

# Schedule tab
output$schedule_table <- renderDT({
  datatable(filtered_data(), options = list(pageLength = 10))
})

# Elo table tab
output$elo_table <- renderDT({
  req(input$week_elo)

  df <- full_data()

  week_num <- as.numeric(input$week_elo)

  all_teams <- df %>%
    select(Home, Visitor) %>%
    pivot_longer(cols = everything(), values_to = "Team") %>%
    distinct(Team)

  get_week_elos <- function(week_num_inner) {
    df_week <- df %>% filter(week <= week_num_inner)

    df_week %>%
      transmute(date,
                Team = Visitor,
                Elo = Visitor_Elo_After) %>%
    bind_rows(df_week %>%
              transmute(date,
                        Team = Home,
                        Elo = Home_Elo_After)) %>%
    arrange(Team, date) %>%
    group_by(Team) %>%
    summarise(Elo = last(Elo), .groups = "drop") %>%
    right_join(all_teams, by = "Team") %>%
    mutate(Elo = round(Elo, 0))
  }

  current_elos <- get_week_elos(week_num) %>%
    mutate(Rank = rank(-Elo, ties.method = "min"))

  if (!is.na(week_num) && week_num > 1) {
    previous_elos <- get_week_elos(week_num - 1) %>%
      mutate(Prev_Rank = rank(-Elo, ties.method = "min")) %>%
      select(Team, Prev_Rank)

    current_elos <- current_elos %>%
      left_join(previous_elos, by = "Team") %>%

```

```

    mutate(
      Change = case_when(
        is.na(Prev_Rank) ~ "-",
        (Prev_Rank - Rank) > 0 ~ paste0("+", Prev_Rank - Rank, " ↑"),
        (Prev_Rank - Rank) < 0 ~ paste0("-", Rank - Prev_Rank, " ↓"),
        TRUE ~ "-"
      )
    )
  } else {
    current_elos <- current_elos %>%
      mutate(Change = "-")
  }

  final_table <- current_elos %>%
    arrange(Rank) %>%
    select(Team, Elo, Change)

  datatable(
    final_table,
    colnames = c("Team", "Elo Rating", "Rank Change"),
    options = list(pageLength = 20),
    rownames = TRUE
  ) %>%
  formatStyle(
    "Change",
    color = styleEqual(
      final_table$Change,
      sapply(final_table$Change, function(change) {
        if (grepl("↑", change)) {
          "green"
        } else if (grepl("↓", change)) {
          "red"
        } else {
          "gray"
        }
      })
    ),
    fontWeight = "bold"
  )
})

}

shinyApp(ui = ui, server = server)

```

Web Scraping Workflow (python_schedule.yml)

```

name: Scrape and Save Schedule

on:
  schedule:
    - cron: '0 12 * * 1' # Runs every Monday at 12:00 UTC
  workflow_dispatch:

jobs:

```

```

scrape:
  runs-on: ubuntu-latest
  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.9'

    - name: Install dependencies
      run: pip install pandas beautifulsoup4 requests lxml

    - name: Run scraping script
      run: python python_schedule_scrape.py

    - name: Commit and push changes
      run: |
        git config --global user.name 'github-actions'
        git config --global user.email 'github-actions@github.com'
        git add schedule_2425.csv
        git commit -m 'Updated schedule data' || echo "No changes to
commit"
        git push

```

Python Web Scraping (python_schedule_scrape.py)

```

import requests
import pandas as pd
from bs4 import BeautifulSoup
from datetime import datetime, timedelta

def scrape_results_function(date):
    date_code = date.strftime('%Y%m%d')
    url =
f"https://www.collegehockeynews.com/women/schedule.php?date={date_code}"
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')

    table = soup.find('table')
    if table is None:
        return pd.DataFrame()

    df = pd.read_html(str(table))[0]
    return clean_sched_function(df)

def clean_sched_function(df):
    df = df.iloc[:, [0, 1, 3, 4, 5]] # Select columns
    df.columns = ['Visitor', 'Visitor_Score', 'Home', 'Home_Score', 'OT']

    df = df[df['Home'].notna() & (df['Home'].str.strip() != '')]
    df['Date_string'] = df['Visitor'].where(df['Visitor'].str.match(r'^[A-Za-
z]+, [A-Za-z]+ \d{1,2}, \d{4}$'))
    df['Date_string'] = df['Date_string'].fillna(method='ffill')

```

```

df = df.dropna(subset=['Date_string'])

df['Date'] = pd.to_datetime(df['Date_string'], format='%A, %B %d, %Y')
df['Division'] = df['Visitor'].where(df['Visitor'] ==
df['Visitor_Score'])
df['Division'] = df['Division'].fillna(method='ffill')

df = df[(df['Visitor'] != df['Visitor_Score']) & (df['Division'] !=
'Exhibition')]
df = df.drop(columns=['Date_string'])
df['OT'] = df['OT'].apply(lambda x: 1 if x == 'ot' else 0)

return df[['Date', 'Visitor', 'Visitor_Score', 'Home', 'Home_Score',
'OT', 'Division']]

def main():
    start_date = datetime(2024, 9, 28)
    end_date = datetime(2025, 3, 31)
    dates = [start_date + timedelta(days=i) for i in range(0, (end_date -
start_date).days + 1, 8)]

    schedule = scrape_results_function(datetime(2024, 9, 20))
    for date in dates:
        temp = scrape_results_function(date)
        schedule = pd.concat([schedule, temp], ignore_index=True)

    schedule.to_csv('schedule_2425.csv', index=False)

if __name__ == "__main__":
    main()

```